

Guida Completa di Studio

Sistemi Intelligenti

Basata sugli appunti del corso (75 pagine)

A.A. 2025–2026

Indice

1	Introduzione all'Intelligenza Artificiale	4
1.1	Cos'è l'IA e i quattro approcci	4
1.2	Fondamenti multidisciplinari	4
1.3	Breve storia dell'IA	5
1.4	Agenti intelligenti	5
2	Ricerca Non Informata (Blind Search)	6
2.1	Formulazione del problema	6
2.2	Albero di ricerca e frontiera	6
2.3	Ricerca in Ampiezza (BFS)	6
2.4	Ricerca a Costo Uniforme (UCS / Dijkstra)	7
2.5	Ricerca in Profondità (DFS)	7
2.6	Ricerca a Profondità Limitata e Approfondimento Iterativo	7
2.7	Ricerca Bidirezionale	8
2.8	Tabella di confronto ricerca non informata	8
3	Ricerca Informata (Heuristic Search)	8
3.1	Funzioni euristiche	8
3.2	Ricerca Greedy Best-First	8
3.3	A*	9
3.4	A* pesata e ricerca soddisfacente	9
3.5	Algoritmi con memoria limitata	9
3.6	Ricerca bidirezionale euristica	10
3.7	Euristiche: problemi rilassati e pattern database	10
4	Ricerca Avversariale (Adversarial Search)	10

4.1	Giochi a somma zero	10
4.2	Algoritmo Minimax	11
4.3	Potatura Alpha-Beta	11
4.4	Tecniche avanzate per la ricerca nei giochi	12
5	Problemi di Soddisfacimento di Vincoli (CSP)	13
5.1	Definizione	13
5.2	Propagazione dei vincoli: consistenza locale	13
5.3	Ricerca con Backtracking	14
5.4	Inferenza durante la ricerca	15
5.5	Backjumping con conflitti	15
6	Agenti Logici e Logica Proposizionale	15
6.1	Agenti basati sulla conoscenza	15
6.2	Concetti fondamentali di logica	16
6.3	Logica proposizionale: sintassi e semantica	16
6.4	Inferenza tramite Model Checking	17
6.5	Dimostrazione di teoremi	17
6.6	Forma Normale Congiuntiva (CNF) e risoluzione	17
6.7	Clausole di Horn e Forward/Backward Chaining	17
7	Logica del Primo Ordine (FOL)	18
7.1	Sintassi della FOL	18
7.2	Quantificatori	18
7.3	Semantica e modelli FOL	19
7.4	Uguaglianza e query	19
8	Inferenza nella FOL	20
8.1	Proposizionalizzazione e teorema di Herbrand	20
8.2	Unificazione	20
8.3	Modus Ponens Generalizzato (Lifted)	20
8.4	Forward e Backward Chaining nella FOL	21
8.5	Risoluzione nella FOL e CNF	21
9	Rappresentazione della Conoscenza e Ontologie	21
9.1	Ingegneria ontologica	21
9.2	Categorie e oggetti	22
9.3	Eventi, fluenti e tempo	22
9.4	Situation Calculus e pianificazione	22
9.5	Relazioni tra ontologie	23

10 Apprendimento Automatico (Machine Learning)	23
10.1 Paradigmi	23
10.2 Valutazione dei modelli	23
10.3 Alberi di decisione	24
11 Reti Neurali	25
11.1 Percettrone	25
11.2 Passata Forward e Backward	25
11.3 MLP – Multi-Layer Perceptron	25
11.4 Backpropagation	26
12 Apprendimento per Rinforzo	27
12.1 Framework MDP	27
12.2 Policy e Value Function	27
12.3 Equazione di Bellman	27
12.4 Algoritmi	28
13 Riepilogo finale	28
13.1 Formule chiave da ricordare	28
13.2 Tabella di confronto degli algoritmi di ricerca	29
13.3 Timeline storica dell'IA	29

1 Introduzione all'Intelligenza Artificiale

1.1 Cos'è l'IA e i quattro approcci

Quattro definizioni di Intelligenza Artificiale

Non esiste una definizione unica. Il campo si divide in quattro approcci:

Approccio	Descrizione	Focus
Agire umanamente	Test di Turing: sistema “intelligente” se simula un umano	Comportamento esterno
Pensare umanamente	Modellazione cognitiva dei processi mentali umani	Funzionamento interno
Pensare razionalmente	Uso della logica per ragionare correttamente	Logica e ragionamento
Agire razionalmente	Agente che massimizza il risultato atteso	Obiettivi e razionalità

Il testo adotta l'approccio “**agire razionalmente**”: costruire agenti che massimizzano la misura di prestazione data la sequenza di percezioni.

Test di Turing (1950)

Un giudice umano conversa via tastiera con due interlocutori: uno umano, uno macchina. Se non riesce a distinguerli, la macchina supera il test.

Capacità richieste: **NLP** (comunicare in linguaggio naturale), **rappresentazione della conoscenza**, **ragionamento automatico**, **apprendimento automatico**, **visione artificiale**, **robotica**.

IA Forte vs IA Debole

- **IA Debole (Weak AI)**: la macchina *simula* l'intelligenza per compiti specifici; non ha comprensione reale.
- **IA Forte (Strong AI)**: la macchina ha vera comprensione, coscienza, stati mentali. Equivalente all'intelligenza umana.
- **Argomento della Stanza Cinese** (Searle): seguire regole sintattiche non implica comprensione semantica. Un operatore che traduce simboli cinesi seguendo un manuale non *capisce* il cinese.

1.2 Fondamenti multidisciplinari

L'IA si basa su: **Filosofia** (logica, razionalità, natura del pensiero – Aristotele), **Matematica** (logica formale, probabilità, NP-completezza), **Economia** (utilità, teoria dei giochi), **Neuroscienze** (base delle reti neurali), **Psicologia** (cognitivismo, modellazione degli agenti), **Informatica** (Legge di Moore), **Cibernetica** (sistemi autoregolanti, feedback – Wiener), **Linguistica** (NLP).

1.3 Breve storia dell'IA

Periodo	Eventi principali
1943–1956	Neuroni artificiali (McCulloch & Pitts), Test di Turing
1952–1969	Logic Theorist, General Problem Solver, Lisp, primi giochi
1969–1986	Sistemi esperti, crisi e “inverno dell'IA”
Dal 1986	Reti neurali + backpropagation (Rumelhart et al.)
Dal 1987	Metodi statistici, reti bayesiane, HMM
2001–oggi	Big Data, deep learning, grandi modelli linguistici

1.4 Agenti intelligenti

Agente e razionalità

Un **agente** è qualsiasi entità capace di **percepire** l'ambiente tramite sensori e **agire** tramite attuatori. Il **programma agente** mappa la sequenza percettiva alle azioni.

Un agente è **razionale** se sceglie l'azione con le migliori conseguenze attese in base a: sequenza percettiva, conoscenze disponibili, azioni possibili, e **misura di prestazione**.

Razionalità $\neq$ onniscienza: l'onniscienza non è realizzabile in pratica (esistono sempre fattori ignoti).

Concetti associati: **Apprendimento** (adattarsi ad ambienti nuovi) e **Autonomia** (affidarsi sempre meno alla conoscenza iniziale e più all'esperienza).

Cinque tipi di agenti

1. **Reattivo semplice**: regole condizione-azione, nessuna memoria. Veloce ma limitato.
2. **Reattivo con modello**: tiene uno stato interno del mondo; funziona in ambienti dinamici/parzialmente osservabili.
3. **Basato su obiettivi (goal-driven)**: pianifica sequenze di azioni per raggiungere scopi.
4. **Basato sull'utilità (utility-driven)**: massimizza una funzione di utilità; gestisce trade-off e incertezza.
5. **Capace di apprendere**: modifica il comportamento tramite esperienza. Composto da: *critico* (valuta le prestazioni), *modulo di apprendimento*, *generatore di problemi* (azioni esplorative).

Caratteristiche degli ambienti

- Completamente/Parzialmente osservabile
- Deterministico / Stocastico
- Episodico / Sequenziale
- Statico / Dinamico
- Discreto / Continuo
- Singolo agente / Multiagente

Ambiente deterministico: si usa un sistema ad **anello aperto** (esegue piano senza rileggere l'ambiente). **Ambiente non deterministico**: si usa un sistema ad **anello chiuso** (mantiene le percezioni e le usa per riadattarsi).

2 Ricerca Non Informata (Blind Search)

2.1 Formulazione del problema

Componenti di un problema di ricerca

- **Spazio degli stati:** tutte le configurazioni possibili.
- **Stato iniziale:** s_0 .
- **Azioni (operatori):** mosse possibili in ogni stato.
- **Modello di transizione:** $\text{Result}(s, a)$ – effetto di a in s .
- **Test obiettivo:** $\text{IsGoal}(s)$.
- **Costo del cammino:** somma dei costi dei singoli passi.

Una **soluzione** è una sequenza di azioni che porta da s_0 a uno stato obiettivo.

2.2 Albero di ricerca e frontiera

Gli algoritmi sovrappongono allo spazio degli stati un **albero di ricerca**: i nodi sono stati, i rami azioni. La **frontiera** (fringe) è l'insieme dei nodi generati ma non ancora espansi. Espandere un nodo n significa applicare tutte le azioni disponibili e generare i nodi figli.

Metriche di valutazione degli algoritmi

- **Completezza:** Trova sempre una soluzione se esiste?
- **Ottimalità:** Trova la soluzione di costo minimo?
- **Complessità temporale:** Nodi generati/espansi.
- **Complessità spaziale:** Memoria massima occupata.

Parametri: b = fattore di ramificazione, d = profondità soluzione ottima, m = profondità massima albero.

2.3 Ricerca in Ampiezza (BFS)

BFS – Breadth-First Search

Espande prima la radice, poi tutti i figli al livello 1, poi al livello 2, ... Usa una coda **FIFO**.

- **Completo:** Sì (se b finito)
- **Ottimale:** Sì (se tutti i costi sono uguali)
- **Tempo:** $O(b^{d+1})$
- **Spazio:** $O(b^{d+1})$ – il collo di bottiglia principale

Limite: consuma molta memoria poiché mantiene tutti i nodi del livello corrente.

2.4 Ricerca a Costo Uniforme (UCS / Dijkstra)

UCS – Uniform Cost Search

Espande sempre il nodo con costo cumulativo $g(n)$ minore. Usa una coda con priorità ordinata per g .

- **Completo:** Sì (se costi $\geq \varepsilon > 0$)
- **Ottimale:** Sì
- **Tempo/Spazio:** $O(b^{1+\lceil C^*/\varepsilon \rceil})$, dove C^* = costo ottimo

Algoritmo di Dijkstra: variante senza obiettivo specifico; calcola i cammini minimi da una sorgente a tutti i nodi (continua finché la frontiera non è vuota). Le “linee di confine” di UCS sono cerchi concentrici intorno alla radice.

2.5 Ricerca in Profondità (DFS)

DFS – Depth-First Search

Espande sempre il nodo più profondo nella frontiera. Usa uno stack LIFO.

- **Completo:** No (può ciclare in grafi infiniti; Sì su alberi finiti con controllo cicli)
- **Ottimale:** No
- **Tempo:** $O(b^m)$
- **Spazio:** $O(b \cdot m)$ – vantaggio principale

Mantiene in memoria solo il cammino dalla radice al nodo corrente + i fratelli non espansi.

2.6 Ricerca a Profondità Limitata e Approfondimento Iterativo

Ricerca a Profondità Limitata (DLS)

DFS con un limite massimo di profondità l . Non completo se la soluzione è a profondità $> l$. Non ottimale.

IDDFS – Iterative Deepening DFS

Esegue ripetutamente DLS con limite $l = 0, 1, 2, \dots$ fino a trovare la soluzione.

- **Completo:** Sì
- **Ottimale:** Sì (se costi uniformi)
- **Tempo:** $O(b^d)$
- **Spazio:** $O(b \cdot d)$

Overhead di ripetizione: i nodi dei livelli più superficiali vengono riesplorati, ma il costo aggiuntivo è circa $\frac{b}{b-1}$ – trascurabile per b grande. **Migliore strategia blind** in termini di tempo + spazio.

2.7 Ricerca Bidirezionale

Ricerca Bidirezionale

Due ricerche simultanee: una in avanti da s_0 , una all'indietro dall'obiettivo. Si fermano quando le frontiere si incontrano.

- **Tempo/Spazio:** $O(b^{d/2})$ – riduzione esponenziale
- **Problema:** serve la funzione di successione inversa; il test di incontro è complesso.
- **Completo e Ottimale:** Sì, se entrambe usano strategie corrette (es. BFS o UCS).

2.8 Tabella di confronto ricerca non informata

Algoritmo	Completo	Ottimale	Tempo	Spazio
BFS	Sì	Sì*	$O(b^{d+1})$	$O(b^{d+1})$
DFS	No	No	$O(b^m)$	$O(bm)$
DLS	No	No	$O(b^l)$	$O(bl)$
IDDFS	Sì	Sì*	$O(b^d)$	$O(bd)$
UCS	Sì	Sì	$O(b^{1+C^*/\varepsilon})$	$O(b^{1+C^*/\varepsilon})$
Bidirezionale	Sì	Sì*	$O(b^{d/2})$	$O(b^{d/2})$

*Con costi uniformi.

3 Ricerca Informata (Heuristic Search)

3.1 Funzioni euristiche

Euristica $h(n)$

Funzione che **stima** il costo del cammino ottimo da n al goal.

- **Ammissibile:** $h(n) \leq h^*(n)$ per ogni n (non sovrastima mai il costo reale).
- **Consistente (monotona):** $h(n) \leq c(n, n') + h(n')$ per ogni arco (n, n') .

Consistente $\Rightarrow$ Ammissibile. Più h è accurata, meno nodi si esplorano.

3.2 Ricerca Greedy Best-First

Esponde il nodo con $h(n)$ minima (ignora g). Veloce ma non ottimale e non completa. Può restare intrappolata in vicoli ciechi. Complessità: $O(b^m)$ nel caso peggiore.

Esempio: Mappa della Romania

Con la distanza in linea d'aria come h , Greedy può scegliere città vicine a Bucharest in linea d'aria ma non lungo il percorso a costo minimo.

3.3 A*

Algoritmo A*

Ordina la frontiera per $f(n) = g(n) + h(n)$:

- $g(n)$: costo effettivo da s_0 a n
- $h(n)$: stima euristica da n al goal
- $f(n)$: stima del costo totale del cammino ottimo passante per n

A* è una **fusione** tra UCS (che usa solo g) e Greedy (che usa solo h).

Proprietà chiave:

1. **Completo:** Sì
2. **Ottimale:** Sì, se h è *ammissibile* (alberi) o *consistente* (grafi con lista chiusa)
3. **Ottimalmente efficiente:** nessun algoritmo ottimale espande meno nodi di A* (a parità di euristica)

A* esegue potatura: scarta nodi il cui valore f indica che non portano a soluzioni ottime, anche se sono figli di nodi espansi. Le “linee di confine” di A* sono ellissi orientate verso il goal (non cerchi concentrici come UCS).

3.4 A* pesata e ricerca soddisfacente

Quando trovare la soluzione ottimale è troppo costoso, si usa:

Weighted A* (A* pesata)

$$f(n) = g(n) + W \cdot h(n), \quad W > 1$$

Dà più importanza all'euristica $\Rightarrow$ esplora meno nodi ma può perdere l'ottimalità. Rimane completo se W è finito e i costi sono positivi.

Euristica inammissibile: può sovrastimare il costo $\Rightarrow$ perde la garanzia di ottimalità ma trova soluzioni più velocemente.

3.5 Algoritmi con memoria limitata

RBFS – Recursive Best-First Search

Simula A* in modo ricorsivo mantenendo solo il cammino corrente + il miglior f alternativo degli antenati. Se f del nodo corrente supera la soglia, backtrack e aggiorna il genitore.

- **Spazio:** $O(bd)$ – lineare
- **Limite:** può ripetere lavoro già fatto (rigenerazione di nodi)
- **Completo e Ottimale:** Sì (con h ammissibile)

SMA* – Simplified Memory-bounded A*

Quando la memoria è piena, rimuove il nodo con f più alto (il peggiore), salvando il suo valore nel genitore per non perdere l'informazione.

- **Completo:** Sì se la memoria basta per il cammino alla soluzione
- **Ottimale:** Sì se la memoria basta per il cammino ottimale
- Può dover rigenerare nodi rimossi: aumenta il tempo di ricerca

3.6 Ricerca bidirezionale euristica

Ricerca bidirezionale euristica

Due ricerche informate: avanti $f_F(n) = g_F(n) + h_F(n)$, indietro $f_B(n) = g_B(n) + h_B(n)$.
Vantaggio: da $O(b^d)$ a $O(b^{d/2})$ nodi. **Difficoltà:** garantire l'ottimalità è complesso – bisogna minimizzare $\max(f_F(n), f_B(n))$ o $f_F(n) + f_B(n)$ per scegliere quale nodo espandere.

3.7 Euristiche: problemi rilassati e pattern database

Come costruire buone euristiche

Problema rilassato: versione semplificata con meno vincoli $\Rightarrow$ il costo ottimale nel problema rilassato è un'euristica **ammissibile** per il problema originale.

Esempio (8-puzzle):

- h_1 = pezzi fuori posto (ignora posizioni)
- h_2 = distanza di Manhattan $= \sum_i |x_i - x_i^*| + |y_i - y_i^*|$ (ignora ostacoli)

h_2 **domina** h_1 : $h_2(n) \geq h_1(n) \forall n \Rightarrow A^*$ espande meno nodi con h_2 .

Pattern Database: si risolve un sottoproblema (es. tasselli 1–4) completamente, si memorizzano i costi in una tabella. Si usano come euristica ammissibile nel problema completo. Pattern *disgiunti* possono essere sommati (euristica additiva ammissibile).

4 Ricerca Avversariale (Adversarial Search)

4.1 Giochi a somma zero

Formulazione di un gioco

Un gioco è definito da:

- s_0 : stato iniziale
- $\text{Player}(s)$: chi muove in s
- $\text{Actions}(s)$: mosse legali in s
- $\text{Result}(s, a)$: stato successivo
- $\text{Terminal}(s)$: test di fine partita
- $\text{Utility}(s, p)$: valore finale per il giocatore p

Gioco a somma zero: ciò che guadagna MAX è perso da MIN. Entrambi hanno **informazione perfetta** (vedono tutto lo stato). Esempio: tris, scacchi, dama.

4.2 Algoritmo Minimax

Minimax

MAX massimizza, MIN minimizza. Backtracking dagli stati terminali:

$$\text{MiniMax}(s) = \begin{cases} \text{Utility}(s) & \text{se } s \text{ terminale} \\ \max_a \text{MiniMax}(\text{Result}(s, a)) & \text{se Player}(s) = \text{MAX} \\ \min_a \text{MiniMax}(\text{Result}(s, a)) & \text{se Player}(s) = \text{MIN} \end{cases}$$

Strategia: MAX deve essere preveggenete – assumere che MIN giochi in modo ottimale.

- **Completo:** Sì (albero finito)
- **Ottimale:** Sì (contro avversario ottimale)
- **Tempo:** $O(b^m)$
- **Spazio:** $O(bm)$

Per gli scacchi: $b \approx 35$, $m \approx 80 \Rightarrow 35^{80}$ nodi – impraticabile senza potatura.

Minimax per giochi multiplayer

Con più di 2 giocatori, ogni nodo ha un **vettore di utilità** $(v_A, v_B, v_C, \dots)$. Ogni giocatore al proprio turno massimizza la propria componente del vettore. Questo può portare ad alleanze e coalizioni.

4.3 Potatura Alpha-Beta

Alpha-Beta Pruning

Ottimizzazione di Minimax che elimina rami irrilevanti mantenendo due valori:

- α : il miglior valore garantito a MAX lungo il cammino corrente (lower bound per MAX)
- β : il miglior valore garantito a MIN lungo il cammino corrente (upper bound per MIN)

Potatura β (in nodo MIN): se il valore scende $\leq \alpha$, pota – MAX non sceglierà mai questo ramo.

Potatura α (in nodo MAX): se il valore sale $\geq \beta$, pota – MIN non sceglierà mai questo ramo.

Complessità:

- **Caso migliore** (ordine perfetto delle mosse): $O(b^{m/2})$ – raddoppia la profondità esplorabile
- **Caso medio** (ordine casuale): $O(b^{3m/4})$
- **Caso peggiore** (ordine inverso): $O(b^m)$

Il risultato è **identico a Minimax**: la potatura non cambia la decisione.

4.4 Tecniche avanzate per la ricerca nei giochi

Ordinamento delle mosse e Tabelle di Trasposizione

Move ordering (migliora la potatura α - β):

- **Approfondimento iterativo:** ricerche a profondità crescenti; le mosse migliori trovate vengono provate per prime nelle ricerche successive.
- **Euristiche killer:** dare priorità a mosse che hanno causato potature significative in passato alla stessa profondità.

Tabelle di trasposizione: memorizzano il valore di ogni stato già visitato. Se uno stato viene raggiunto da percorsi diversi, il valore viene recuperato dalla tabella (accesso in $O(1)$) evitando di riesaminare tutto il sottoalbero.

Funzione di Valutazione (EVAL)

Quando la ricerca è limitata in profondità si applica agli stati non terminali una funzione di valutazione euristica:

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

Combinazione lineare di **feature** f_i con pesi w_i . Le feature devono essere normalizzate. Esempio (scacchi): pedone=1, cavallo/alfiere=3, torre=5, regina=9. Feature avanzate: controllo del centro, sicurezza del re, mobilità.

Differenza con la funzione di utilità: la Utility è esatta e si usa solo negli stati terminali. EVAL è una stima euristica usata negli stati non terminali.

Effetto orizzonte e ricerca di quiescenza

Effetto orizzonte: l'algoritmo non vede minacce imminenti oltre il limite di profondità (un pezzo perso che "si nasconde" oltre l'orizzonte con mosse dilatorie).

Ricerca di quiescenza: continuare la ricerca oltre il limite solo in posizioni "turbolente" (prese, scacchi, mosse che cambiano drasticamente la situazione) fino a trovare uno **stato quiescente** (stabile) dove EVAL è affidabile.

Potatura in avanti (Forward Pruning)

Scarta in anticipo rami deboli **senza calcolare il valore esatto** (non garantisce ottimalità):

- **Beam Search:** esplora solo le N mosse migliori per nodo secondo EVAL. Rischio: scartare la mossa vincente.
- **ProbCut:** stima probabilisticamente se un ramo può migliorare α/β ; pota se la probabilità è bassa.
- **Late Move Reduction:** riduce la profondità di ricerca per le mosse "tardive" (meno promettenti).

5 Problemi di Soddisfacimento di Vincoli (CSP)

5.1 Definizione

CSP – Constraint Satisfaction Problem

Un CSP è una tripla $\langle X, D, C \rangle$:

- $X = \{X_1, \dots, X_n\}$: variabili
- $D = \{D_1, \dots, D_n\}$: domini (un dominio per variabile)
- $C = \{C_1, \dots, C_m\}$: vincoli (coppie $\langle$ ambito, relazione $\rangle$)

Assegnamento consistente (legale): non viola alcun vincolo. **Assegnamento completo:** tutte le variabili hanno un valore. **Soluzione:** completo e consistente.

I CSP potano rapidamente grandi porzioni dello spazio di ricerca: appena un assegnamento parziale viola un vincolo, si scartano tutte le sue estensioni.

Tipologie di vincoli

- **Unari:** su una variabile (es. WA = rosso)
- **Binari:** su due variabili (es. WA $\neq$ NT) – rappresentabili come grafo
- **Globali:** su un numero arbitrario di variabili; si trattano con algoritmi dedicati

Vincoli globali importanti: TutteDiverse (Alldiff) – usato in Sudoku, 8-regine; AlPiù, AlMeno, Esattamente; vincolo **disgiuntivo** (due attività non si sovrappongono nel tempo).

Vincoli di preferenza (soft constraints): Constraint Optimization Problems (COP) – trovare la soluzione che massimizza la soddisfazione dei vincoli morbidi.

Esempi di CSP

- **Colorazione mappa Australia:** variabili = regioni, dominio = {rosso, verde, blu}, vincoli = regioni adiacenti di colore diverso.
- **8-regine:** 8 variabili (colonne), dominio = {1..8} (riga), vincoli = nessuna coppia di regine sulla stessa riga, colonna, diagonale.
- **Sudoku:** 81 variabili, dominio = {1..9}, 27 vincoli TutteDiverse (righe, colonne, blocchi 3×3). NP-completo ma molte istanze risolvibili solo per inferenza.

5.2 Propagazione dei vincoli: consistenza locale

Consistenza di nodo (1-consistenza)

Una variabile è **nodo-consistente** se tutti i valori nel suo dominio soddisfano i vincoli unari. Si applica rimuovendo i valori invalidi dai domini.

Consistenza d'arco (2-consistenza) e AC-3

X_i è **arco-consistente** rispetto a X_j se per ogni valore $v \in D_i$ esiste almeno un valore $w \in D_j$ compatibile con il vincolo C_{ij} .

AC-3: gestisce una coda di archi. Quando si rimuove un valore da D_i , si reinseriscono tutti gli archi (X_k, X_i) nella coda (la loro consistenza potrebbe essere stata invalidata).

Complessità AC-3: $O(n^2 d^3)$

dove n = variabili, d = dimensione massima dominio.

Consistenza di cammino (3-consistenza)

Una coppia (X_i, X_j) è **cammino-consistente** rispetto a X_m se per ogni assegnamento $\{X_i = a, X_j = b\}$ consistente esiste un valore per X_m che soddisfa C_{im} e C_{mj} .

Permette di eliminare combinazioni di valori che non possono estendersi a soluzioni valide, riducendo i domini in modo più mirato di AC.

K-consistenza

Un CSP è **k-consistente** se per ogni insieme di $k - 1$ variabili con assegnamento consistente, esiste un valore per qualsiasi k -esima variabile compatibile con quelle assegnate.

Gerarchia: 1-consistenza = consistenza di nodo; 2-consistenza = arc consistency; 3-consistenza $\approx$ consistenza di cammino.

Forte k-consistenza: i-consistente per ogni $i \leq k$. Se forte n -consistente $\Rightarrow$ soluzione senza backtracking. **Costo:** $O(n^k d^k)$ – proibitivo per k grande; in pratica si usa al massimo $k = 2$ o 3 .

5.3 Ricerca con Backtracking

Backtracking Search per CSP

Assegna variabili una alla volta; se un assegnamento viola un vincolo, lo scarta e prova il valore successivo; se nessun valore è valido, torna alla variabile precedente.

Euristiche per la scelta della variabile:

- **MRV (Minimum Remaining Values):** scegliere la variabile con il dominio più piccolo (principio *fail-first* – scoprire prima i fallimenti).
- **Euristica di grado (Degree Heuristic):** tra le variabili con MRV uguale, scegliere quella con più vincoli sulle variabili non assegnate (riduce il fattore di ramificazione futuro).

Euristica per la scelta del valore:

- **Least Constraining Value:** assegnare il valore che elimina meno valori dai domini delle variabili adiacenti (lascia più opzioni aperte).

5.4 Inferenza durante la ricerca

Forward Checking (Verifica in avanti)

Dopo ogni assegnamento, si eliminano dai domini delle variabili vicine non assegnate i valori incompatibili. Se un dominio si svuota $\Rightarrow$ backtrack immediato (senza esplorare ulteriormente).

Mantiene la **consistenza d'arco locale** (solo gli archi che toccano la variabile appena assegnata).

MAC – Maintaining Arc Consistency

Più potente del forward checking: dopo ogni assegnamento, applica **AC-3** all'intero CSP. Pota più fortemente ma è più costoso computazionalmente.

5.5 Backjumping con conflitti

Conflict-Directed Backjumping

Il backtracking cronologico torna sempre alla variabile precedente, anche se non è la causa del fallimento (inefficiente).

Backjumping: quando una variabile X fallisce (dominio vuoto), si costruisce l'insieme dei conflitti $CS(X)$ – le variabili già assegnate che causano il fallimento – e si torna direttamente all'ultima variabile in $CS(X)$.

Implementazione: ogni livello riceve il conflict set dai figli falliti, aggiunge la propria variabile e lo passa indietro.

Nota: se si usa MAC, molte inconsistenze vengono già rilevate prima, rendendo il backjumping meno necessario.

6 Agenti Logici e Logica Proposizionale

6.1 Agenti basati sulla conoscenza

Knowledge-Based Agent

Il cuore di questi agenti è la **Base di Conoscenza (KB)**: insieme di formule in un linguaggio formale. Ciclo di funzionamento:

1. Percepisce l'ambiente (sensori).
2. **TELL(KB, percezione)**: aggiunge nuove informazioni alla KB.
3. **ASK(KB, query)**: interroga la KB per decidere l'azione ottimale.
4. Esegue l'azione.

Il comportamento può essere **dichiarativo** (regole nella KB) o **procedurale** (codice diretto). Questi agenti possono **apprendere**: aggiornano la KB con l'esperienza.

Il Mondo del Wumpus

Caverne 4×4 con: Wumpus (uccide), Pozzi (uccide), Oro (obiettivo). L'agente parte da (1,1) e deve trovare l'oro e uscire. Ambiente: discreto, deterministico, statico, singolo agente, **parzialmente osservabile**.

Percezioni: Fetore (Wumpus adiacente), Brezza (Pozzo adiacente), Scintillio (oro nella stanza), Urto (muro), Grido (Wumpus ucciso).

Punteggio: +1000 uscire con oro, -1000 morire, -1 per azione, -10 per freccia.

6.2 Concetti fondamentali di logica

Sintassi, Semantica, Conseguenza logica

- **Sintassi:** struttura formale del linguaggio – quali sequenze di simboli sono formule ben formate.
- **Semantica:** significato – quando una formula è vera o falsa in un *modello*.
- **Modello:** stato del mondo possibile; assegna valori di verità alle proposizioni atomiche. $M(\alpha)$ = insieme dei modelli in cui α è vera.
- **Conseguenza logica:** $KB \models \alpha$ se α è vera in **tutti** i modelli in cui KB è vera.
- **Inferenza:** $KB \vdash_i \alpha$ – algoritmo i deriva α da KB.
- **Correttezza:** $KB \vdash_i \alpha \Rightarrow KB \models \alpha$ (non deduce falsità).
- **Completezza:** $KB \models \alpha \Rightarrow KB \vdash_i \alpha$ (non trascuria verità).

6.3 Logica proposizionale: sintassi e semantica

Sintassi della logica proposizionale

Formule atomiche: proposizioni semplici ($P, Q, P_{1,2}, \dots$).

Connettivi logici:

- $\neg P$: negazione – “non P”
- $P \wedge Q$: congiunzione – “P e Q”
- $P \vee Q$: disgiunzione – “P o Q (o entrambi)”
- $P \Rightarrow Q$: implicazione – “se P allora Q” ($\equiv \neg P \vee Q$)
- $P \Leftrightarrow Q$: bicondizionale – “P se e solo se Q”

Validità, Soddisfacibilità, Equivalenza

- **Valida (tautologia):** vera in **tutti** i modelli. Es: $A \vee \neg A$.
- **Soddisfacibile:** vera in **almeno un** modello. SAT è NP-completo.
- **Insoddisfacibile:** falsa in tutti i modelli (contraddizione).
- **Equivalenza logica:** $\alpha \equiv \beta$ se $\alpha \models \beta$ e $\beta \models \alpha$ (stessi modelli).
- **Teorema di deduzione:** $KB \models \alpha \Leftrightarrow (KB \Rightarrow \alpha)$ è valida.
- **Dimostrazione per assurdo:** $KB \models \alpha \Leftrightarrow KB \wedge \neg \alpha$ è insoddisfacibile.

6.4 Inferenza tramite Model Checking

Model Checking

Enumerare tutti i modelli (tutte le combinazioni di verità delle proposizioni). Per ogni modello in cui KB è vera, verificare se α è vera. Se sì per tutti $\Rightarrow KB \models \alpha$.

Complessità: $O(2^n)$ con n proposizioni – esponenziale, impraticabile per basi di conoscenza grandi.

6.5 Dimostrazione di teoremi

Regole di inferenza

Modus Ponens:

$$\frac{p_1 \wedge p_2 \wedge \dots \wedge p_k \quad p_1 \wedge \dots \wedge p_k \Rightarrow q}{q}$$

Eliminazione AND: da $\alpha \wedge \beta$ si deriva α (o β).

Regola di Risoluzione: da $(A \vee C)$ e $(\neg C \vee B)$ si deriva $(A \vee B)$. I letterali complementari C e $\neg C$ vengono eliminati (fattorizzazione).

6.6 Forma Normale Congiuntiva (CNF) e risoluzione

CNF e PL-RISOLUZIONE

CNF (Forma Normale Congiuntiva): congiunzione di *clausole*, dove ogni clausola è una disgiunzione di letterali. Es: $(A \vee \neg B) \wedge (B \vee C \vee \neg A)$.

Conversione in CNF: eliminare $\Leftrightarrow$ e $\Rightarrow$; portare $\neg$ verso l'interno (De Morgan); distribuire $\vee$ su $\wedge$.

PL-RISOLUZIONE (per refutazione): per dimostrare $KB \models \alpha$, si aggiunge $\neg\alpha$ alla KB in CNF e si applicano ripetutamente le risoluzioni. Se si deriva la **clausola vuota** $\square$ (contraddizione) $\Rightarrow \alpha$ è conseguenza logica.

Teorema: se un insieme di clausole è insoddisfacibile, la sua chiusura della risoluzione contiene la clausola vuota.

6.7 Clausole di Horn e Forward/Backward Chaining

Clausole di Horn

Clausola con **al più un letterale positivo**: $\neg P_1 \vee \neg P_2 \vee \dots \vee \neg P_k \vee Q$, ovvero $P_1 \wedge \dots \wedge P_k \Rightarrow Q$.

Tipi:

- **Clausole definite:** esattamente un letterale positivo (implicazione con conseguente).
- **Clausole obiettivo:** nessun letterale positivo – negazione di ciò che si vuole dimostrare.

L'inferenza su clausole di Horn è **decidibile in tempo polinomiale** (forward/backward chaining). **Risoluzione unitaria** (applicare risoluzione quando almeno una clausola è

unitaria): completa per clausole di Horn.

Forward Chaining – Concatenazione in avanti

Guidato dai dati. Parte dai fatti noti; ogni volta che le premesse di una regola sono soddisfatte, aggiunge la conclusione ai fatti noti. Ripete finché trova il goal o non produce nuovi fatti.

Corretto e completo per clausole definite. Può generare fatti irrilevanti per la query.

Backward Chaining – Concatenazione all'indietro

Guidato dall'obiettivo. Parte dal goal; cerca regole la cui conclusione corrisponde al goal; le premesse diventano sotto-obiettivi da dimostrare ricorsivamente.

Corretto e completo per clausole definite. Più efficiente del forward chaining quando la KB è grande (si concentra solo su ciò che serve). **Base di Prolog.**

7 Logica del Primo Ordine (FOL)

7.1 Sintassi della FOL

Elementi della FOL

- **Simboli di costante:** oggetti specifici (es. Riccardo, Giovanni).
- **Simboli di variabile:** $x, y, z, \dots$ – oggetti generici quantificati.
- **Simboli di predicato:** relazioni o proprietà con arità fissa. Es: Fratello(x, y) (arietà 2), Persona(x) (arietà 1).
- **Simboli di funzione:** mappano oggetti in oggetti. Es: GambaSinistra(x), Padre(x).
- **Termini semplici:** costanti o variabili.
- **Termini complessi:** funzione applicata a termini (es. Padre(Riccardo)).
- **Formula atomica:** predicato applicato a termini (es. Fratello(Riccardo, Giovanni)).

7.2 Quantificatori

Quantificatori universale ed esistenziale

Quantificatore universale $\forall$: “per ogni”

$\forall x P(x)$ vera se P vale per ogni oggetto del dominio

Quantificatore esistenziale $\exists$: “esiste almeno un”

$\exists x P(x)$ vera se almeno un oggetto soddisfa P

Relazioni (De Morgan per FOL):

$$\forall x P(x) \equiv \neg \exists x \neg P(x) \quad \exists x P(x) \equiv \neg \forall x \neg P(x)$$

Regola pratica: $\forall$ si abbina con $\Rightarrow$; $\exists$ si abbina con $\wedge$.

Ordine dei quantificatori è fondamentale: $\forall x \exists y \text{ Ama}(x, y) =$ “ognuno ama qualcuno” $\neq \exists y \forall x \text{ Ama}(x, y) =$ “qualcuno è amato da tutti”

7.3 Semantica e modelli FOL

Modelli nella FOL

Un modello FOL è composto da:

1. **Dominio:** insieme non vuoto di oggetti.
2. **Interpretazione:** assegna a ogni costante un oggetto, a ogni predicato una relazione (insieme di tuple), a ogni funzione una funzione totale sul dominio.

Semantica di database: tre ipotesi semplificatrici:

- **Nomi unici:** ogni costante denota un oggetto distinto.
- **Mondo chiuso (CWA):** le formule atomiche sconosciute sono false.
- **Chiusura del dominio:** ogni oggetto è denotato da almeno una costante.

7.4 Uguaglianza e query

Si possono esprimere affermazioni come $\text{Padre}(\text{Giovanni}) = \text{Enrico}$ (i due termini denotano lo stesso oggetto). Con $\neq$: $\exists x \exists y \text{ Fratello}(x, \text{Riccardo}) \wedge \text{Fratello}(y, \text{Riccardo}) \wedge \neg(x = y)$ esprime che Riccardo ha almeno due fratelli distinti.

TELL(KB, formula): aggiunge una formula. **ASK(KB, query):** risponde vero/falso.

ASKVARS(KB, query): restituisce le sostituzioni (lista di legami) per le variabili che rendono la query vera.

8 Inferenza nella FOL

8.1 Proposizionalizzazione e teorema di Herbrand

Da FOL a logica proposizionale

Istanziamento Universale (UI): da $\forall v \alpha$ si può derivare $\text{SUBST}(\{v/g\}, \alpha)$ per qualsiasi termine ground g .

Istanziamento Esistenziale (EI): da $\exists v \alpha$ si deriva $\text{SUBST}(\{v/k\}, \alpha)$ dove k è una nuova costante di Skolem non presente nella KB.

Proposizionalizzazione: applicare UI/EI per ottenere formule ground, poi usare inferenza proposizionale. Se la KB include funzioni, le sostituzioni possibili sono infinite.

Teorema di Herbrand: se $KB \models \alpha$, esiste una dimostrazione che coinvolge solo un sottoinsieme finito delle istanziazioni ground.

8.2 Unificazione

Unificazione

Trovare la sostituzione θ (**MGU – Most General Unifier**) che rende due espressioni sintatticamente identiche: $\text{UNIFY}(x, y) = \theta$ tale che $x\theta = y\theta$.

Esempi:

- $\text{UNIFY}(\text{Conosce}(\text{Giovanni}, x), \text{Conosce}(\text{Giovanni}, \text{Giacomina})) = \{x/\text{Giacomina}\}$
- $\text{UNIFY}(\text{Conosce}(\text{Giovanni}, x), \text{Conosce}(y, \text{Guglielmo})) = \{x/\text{Giovanni}, y/\text{Guglielmo}\}$
- $\text{UNIFY}(\text{Conosce}(\text{Giovanni}, x), \text{Conosce}(x, \text{Elisabetta})) = \text{fallimento}$ (stessa variabile x non può valere due cose)

Standardizzazione separata: rinominare le variabili di ogni clausola per evitare collisioni. L'ultimo caso diventa: $\text{UNIFY}(\text{Conosce}(\text{Giovanni}, x), \text{Conosce}(z, \text{Elisabetta})) = \{x/\text{Elisabetta}, z/\text{Giovanni}\}$.

Occur check: verificare che la variabile non appaia nel termine a cui viene legata (garantisce terminazione).

8.3 Modus Ponens Generalizzato (Lifted)

Modus Ponens Generalizzato

Date premesse $p_1, \dots, p_k$ nella KB e una regola $P_1 \wedge \dots \wedge P_k \Rightarrow Q$ con variabili, se esiste una sostituzione θ tale che $\text{SUBST}(\theta, P_i) = p_i$ per ogni i , si deriva $\text{SUBST}(\theta, Q)$.

Generalizzazione del Modus Ponens alle formule con variabili (lifting).

8.4 Forward e Backward Chaining nella FOL

Forward Chaining FOL (FOL-CA-ASK)

Parte dai fatti noti; fa scattare tutte le regole le cui premesse si unificano con fatti noti; aggiunge le conclusioni. Ripete fino al **punto fisso** (nessun nuovo fatto deducibile).

Corretto e completo per clausole definite. Inefficiente se produce molti fatti irrilevanti. Euristiche MRV applicabile all'ordinamento dei congiunti nelle premesse.

Datalog: clausole definite senza funzioni; più semplice (usato nei database relazionali).

Backward Chaining FOL (FOL-CI-ASK)

Parte dal goal; cerca regole con testa unificabile col goal (standardizzando le variabili); le premesse diventano sotto-obiettivi. Esplorazione depth-first nello spazio degli obiettivi.

Algoritmo: Kowalski: **Algoritmo** = **Logica** + **Controllo**. Implementato da **Prolog** (backward chaining con DFS, ordine dipende dall'ordine delle clausole, CWA).

Limiti: può incorrere in cicli infiniti con regole ricorsive; può ripetere calcoli ridondanti.

8.5 Risoluzione nella FOL e CNF

Conversione in CNF per FOL

1. Eliminare $\Leftrightarrow$ e $\Rightarrow$.
2. Portare $\neg$ verso l'interno (De Morgan, leggi dei quantificatori).
3. Standardizzare variabili (rinominarle per evitare ambiguità).
4. Portare i quantificatori all'esterno (**prenex form**).
5. **Skolemizzazione**: sostituire $\exists x \alpha$ con una costante di Skolem (se $\exists$ è a scope di $\forall y$, usare una funzione di Skolem $f(y)$).
6. Eliminare i $\forall$ (sottintesi).
7. Distribuire $\vee$ su $\wedge$.

La risoluzione FOL è **completa per refutazione**: se $KB \models \alpha$, si può derivare la clausola vuota da $KB \cup \{\neg\alpha\}$.

9 Rappresentazione della Conoscenza e Ontologie

9.1 Ingegneria ontologica

Ontologie

Una **ontologia** è una specifica formale di una concettualizzazione: definisce cosa esiste in un dominio e come i concetti sono collegati.

Componenti: classi/categorie, proprietà/relazioni, istanze, assiomi.

Sfida: equilibrio tra catturare le generalizzazioni principali e gestire le eccezioni.

9.2 Categorie e oggetti

Organizzazione degli oggetti

- **Tassonomia:** gerarchia di categorie (albero). Es: PalloniDaBasket $\subseteq$ Palloni.
- **Disgiunte:** due categorie senza istanze in comune.
- **Decomposizione esaustiva:** ogni istanza della sovracategoria è in almeno una sottocategoria.
- **Partizione:** disgiunte + esaustive.
- **Composizione fisica:** ParteDi(oggetto, entità superiore).
- **Cose vs roba:** oggetti discreti numerabili (“cose”) vs sostanze non numerabili (“roba” – acqua, sabbia). La “roba” si descrive per composizione/misura.

9.3 Eventi, fluenti e tempo

Modellazione del tempo e degli eventi

- **Fluente:** aspetto del mondo la cui verità varia nel tempo. Es: Presidente(USA, t).
- **Azione vs evento:** le azioni sono compiute da un agente; gli eventi accadono (anche senza agente diretto).
- **Relazioni temporali:** Precede, Segue, Durante, Sovrapposti, Consecutivi...
- **Predicati:** Accade(e, t), Inizia(e, t), Finisce(e, t).

9.4 Situation Calculus e pianificazione

Situation Calculus

Rappresentazione logica per ragionare su azioni:

- **Azione:** influenza il mondo.
- **Situazione:** stato risultante dall'esecuzione di azioni.
- **Fluente:** proprietà che cambia valore.
- **Do(Azione, s):** esegue l'azione nella situazione s .

Assiomi:

1. **Applicabilità:** $\forall p, s \text{ Applicable}(a(p), s) \Leftrightarrow \text{Precond}(p, s)$
2. **Effetto:** $\text{Applicable}(a(p), s) \Rightarrow \text{Effects}(p, \text{Result}(a(p), s))$
3. **Stato successore:** un fluente è vero nel risultato $\Leftrightarrow$ (l'azione lo rende vero) OR (era vero e l'azione non lo ha reso falso). Risolve il *frame problem*.

Anomalia di Sussman e Pianificazione

Non sempre è possibile **sequenzializzare** il perseguimento dei sotto-obiettivi: perseguire un sotto-obiettivo può disfare i passi fatti per un altro (Anomalia di Sussman). Soluzione: **interleaving** dei passi di soluzione.

9.5 Relazioni tra ontologie

Tipi di relazione tra ontologie

1. **Identical:** stessa ontologia (es. copia su mirror).
2. **Equivalent:** stesso vocabolario e assiomatizzazione ma linguaggi diversi.
3. **Extension:** O1 estende O2 (tutti i simboli di O2 preservati in O1).
4. **Weakly-Translatable:** si può tradurre con perdita di informazione.
5. **Strongly-Translatable:** vocabolario totalmente mappabile, nessuna perdita.
6. **Approx-Translatable:** weakly + possibili inconsistenze (concetti affini ma non identici).

10 Apprendimento Automatico (Machine Learning)

10.1 Paradigmi

Tipi di apprendimento

- **Supervisionato:** dataset $\{(x_i, y_i)\}$; si apprende $f : x \mapsto y$.
- **Non supervisionato:** solo $\{x_i\}$; si trovano strutture (clustering, riduzione dimensionale).
- **Per rinforzo:** agente in ambiente; impara tramite segnali di reward.

Modello predittivo: predice la classe di istanze nuove. **Modello descrittivo:** evidenzia le caratteristiche che distinguono le categorie.

10.2 Valutazione dei modelli

Matrice di confusione e metriche

La **matrice di confusione** confronta previsioni del modello con le etichette reali (verità).
Celle: TP, FP, TN, FN.

Accuratezza: $\frac{TP+TN}{TP+TN+FP+FN}$ (percentuale di previsioni corrette).

Error rate: $1 - \text{Accuratezza}$.

Matrice dei costi: estensione che pesa diversamente i tipi di errore.

Overfitting e Underfitting

- **Overfitting (alta varianza):** modello troppo specifico; si adatta al rumore del training set; generalizza male. Causa: noise o pochi esempi.
- **Underfitting (alto bias):** modello troppo semplice; non cattura il pattern.
- **Rasoio di Occam:** a parità di errore, i modelli più semplici sono preferibili.
- **Soluzioni:** validazione incrociata, regolarizzazione, pruning (per alberi).

10.3 Alberi di decisione

Alberi di decisione

Strumento per classificazione e decisione. I **nodi interni** rappresentano test su attributi; le **foglie** rappresentano classi. L'istanza viene classificata percorrendo l'albero dal nodo radice.

Algoritmo di Hunt (ID3/C4.5)

Costruzione ricorsiva: dato il sottoinsieme D_t associato al nodo t :

1. Se tutte le istanze appartengono alla stessa classe $\Rightarrow$ foglia.
2. Scegliere l'attributo che effettua il miglior **split** (massimizza il guadagno di informazione).
3. Creare un nodo figlio per ogni valore dell'attributo; assegnare le istanze ai figli.
4. Applicare ricorsivamente.

Se una combinazione di valori non è rappresentata $\Rightarrow$ foglia con classe di default. Se istanze identiche hanno classi diverse (non-determinismo) $\Rightarrow$ foglia con la classe più frequente.

Misure di impurità

La **bontà dello split** si misura con il **guadagno di informazione**:

$$\text{Gain} = \text{Impurità}(t) - \sum_v \frac{|D_v|}{|D_t|} \cdot \text{Impurità}(D_v)$$

Entropia (misura di impurità):

$$H(t) = - \sum_{i=0}^{c-1} p(i|t) \log_2 p(i|t)$$

$H = 0$ se il nodo è puro; $H = 1$ se massima confusione (2 classi equiprobabili).

Information Gain: usa l'entropia come impurità. Favorisce attributi con molti valori $\Rightarrow$ attenzione agli identificatori univoci!

Indice di Gini: $\text{Gini}(t) = 1 - \sum_i p(i|t)^2$.

Pruning degli alberi di decisione

- **Pre-Pruning:** si interrompe la costruzione prima del completamento (se il gain è sotto soglia). Rischio: underfitting.
- **Post-Pruning:** si costruisce l'albero completo poi si potano i rami (si sostituisce un sottoalbero con una foglia etichettata con la classe più frequente nelle sue foglie).

11 Reti Neurali

11.1 Percettrone

Percettrone (Rosenblatt, 1957)

Elemento computazionale singolo: calcola $y = f(\mathbf{w} \cdot \mathbf{x} + b)$ dove f è la funzione di attivazione.

Funzione gradino: $y = 1$ se $\mathbf{w} \cdot \mathbf{x} \geq \theta$, altrimenti 0.

Funzione sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-\beta(z-\theta)}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

dove θ è la soglia/bias, β controlla la pendenza.

Il percertrone codifica un **test lineare**: i pesi definiscono un iperpiano nello spazio degli input; le istanze sopra l'iperpiano vengono classificate nella classe 1. I pesi sono la **conoscenza** del percertrone: persistenti e aggiornati durante l'addestramento.

Limite: non può apprendere funzioni non linearmente separabili (es. XOR).

11.2 Passata Forward e Backward

Forward Pass (Passata in avanti)

Calcola l'output della rete data l'istanza in input:

1. Riceve il vettore di input $\mathbf{x}$.
2. Pesa ogni input con w_i .
3. Calcola $\text{net} = \sum_i w_i x_i + b$.
4. Applica la funzione di attivazione: $y = f(\text{net})$.

L'output permette la classificazione (0 o 1 per classificazione binaria).

Backward Pass – Regola Delta

Aggiornamento dei pesi in base all'errore $(d - o)$ tra valore desiderato d e output o :

$$w_j(t+1) = w_j(t) + \eta \cdot (d - o) \cdot x_j$$

dove η è il **learning rate**. Minimizza l'errore quadratico medio per discesa del gradiente.

Teorema di Novikoff: se il problema è linearmente separabile, l'apprendimento del percertrone converge alla soluzione.

11.3 MLP – Multi-Layer Perceptron

Architettura MLP

Rete neurale **feed-forward** a strati:

1. **Input layer:** un neurone per ogni feature; funzione identità.
2. **Hidden layer(s):** neuroni con sigmoide. Elaborano la rappresentazione intermedia.

3. **Output layer:** combina i risultati degli hidden; classificazione finale.

Universal Approximation Theorem: un MLP con almeno un hidden layer può approssimare qualsiasi funzione continua.

MLP e classificazione:

- Classe singola: 1 neurone di output (si attiva = classe presente).
- Due classi: 1 neurone di output (attivazione = classe A).
- k classi: k neuroni di output (uno per classe; si attiva quello della classe corretta; gli altri restano a 0).

Apprende in modo supervisionato a partire da un **batch** di esempi (learning set). Ogni passata dell'intero learning set = **epoca di apprendimento**.

11.4 Backpropagation

Discesa del gradiente e Backpropagation

Errore (per i neuroni di output):

$$E = \frac{1}{2} \sum_{i=1}^p (t_i - y_i)^2$$

dove p = neuroni di output, t_i = valore desiderato, y_i = valore prodotto.

Aggiornamento pesi tramite discesa del gradiente:

$$\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}}$$

Algoritmo backpropagation:

1. **Forward pass:** calcola output layer per layer.
2. **Calcola l'errore** ai neuroni di output.
3. **Backward pass:** propaga il gradiente all'indietro usando la **regola della catena**:

$$\delta_j^{(l)} = \sigma'(z_j^{(l)}) \sum_k w_{jk}^{(l+1)} \delta_k^{(l+1)}$$

4. **Aggiornamento:** $w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} + \eta \cdot \delta_j^{(l)} \cdot a_i^{(l-1)}$

Problemi pratici dell'addestramento

- **Vanishing gradient:** con sigmoid/tanh il gradiente si azzerava negli strati profondi. Soluzione: ReLU, batch normalization, connessioni residue.
- **Minimi locali:** la loss non è convessa. In pratica i minimi locali con bassa loss sono frequenti e accettabili.
- **Scelta di η :** troppo grande = instabilità; troppo piccolo = convergenza lenta. Soluzione: Adam, RMSProp.
- **Overfitting:** regolarizzazione (L1/L2), dropout.

12 Apprendimento per Rinforzo

12.1 Framework MDP

Markov Decision Process (MDP)

Formalismo per l'apprendimento per rinforzo: $\langle S, A, T, R, \gamma \rangle$

- S : insieme degli stati
- A : insieme delle azioni
- $T(s, a, s')$: probabilità di transire in s' da s con azione a
- $R(s, a, s')$: reward (rinforzo) ricevuto
- $\gamma \in [0, 1)$: fattore di sconto (discount factor)

Proprietà di Markov: lo stato futuro dipende **solo** dallo stato corrente, non dalla storia.

12.2 Policy e Value Function

Policy, Value Function, Q-value

Policy $\pi : S \rightarrow A$: strategia di comportamento (deterministica o stocastica).

Value function (valore di uno stato sotto π):

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s \right]$$

Il fattore γ sconta i reward futuri (reward immediati valgono di più).

Q-value (valore di un'azione in uno stato):

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s, A_0 = a \right]$$

La policy ottimale: $\pi^*(s) = \arg \max_a Q^*(s, a)$.

12.3 Equazione di Bellman

Equazione di Bellman

Il valore ottimale V^* soddisfa:

$$V^*(s) = \max_{a \in A} \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \cdot V^*(s')]$$

Per i Q-values:

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \cdot \max_{a'} Q^*(s', a')]$$

12.4 Algoritmi

Value Iteration

Inizializza $V(s) = 0$ per tutti gli stati. Itera fino a convergenza:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \cdot V_k(s')]$$

Converge a V^* per $\gamma < 1$ (contrazione).

Policy Iteration

Alterna due fasi:

1. **Policy Evaluation:** calcola V^{π_k} risolvendo il sistema lineare.
2. **Policy Improvement:** $\pi_{k+1}(s) \leftarrow \arg \max_a \sum_{s'} T(s, a, s') [R + \gamma V^{\pi_k}(s')]$

Converge più velocemente di Value Iteration ma ogni valutazione è costosa.

Q-Learning (model-free)

Non richiede la conoscenza di T e R . Aggiornamento online dall'esperienza (s, a, r, s') :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

dove α è il learning rate. **Off-policy:** converge a Q^* con sufficiente esplorazione, indipendentemente dalla policy di comportamento.

Esplorazione vs Sfruttamento

ϵ -greedy: con probabilità ϵ scegliere un'azione casuale (**esplorazione**); con probabilità $1 - \epsilon$ scegliere $\arg \max_a Q(s, a)$ (**sfruttamento**). ϵ decresce nel tempo per passare dall'esplorazione allo sfruttamento.

13 Riepilogo finale

13.1 Formule chiave da ricordare

Formulario essenziale

Concetto	Formula / Proprietà
A*: funzione di valutazione	$f(n) = g(n) + h(n)$
Ammissibilità euristica	$h(n) \leq h^*(n)$ per ogni n
Consistenza (monotonicità)	$h(n) \leq c(n, n') + h(n')$
Dist. di Manhattan (griglia)	$\sum_i x_i - x_i^* + y_i - y_i^* $
Minimax MAX	$\max_a \text{MiniMax}(\text{Result}(s, a))$
Minimax MIN	$\min_a \text{MiniMax}(\text{Result}(s, a))$
Funzione di valutazione	$\text{EVAL}(s) = \sum_i w_i f_i(s)$
AC-3 complessità	$O(n^2 d^3)$
Modus Ponens	$\frac{\alpha \quad \alpha \Rightarrow \beta}{\beta}$
Risoluzione	da $(A \vee C), (\neg C \vee B)$ deriva $(A \vee B)$
Sigmoide	$\sigma(z) = \frac{1}{1+e^{-z}}$
Regola Delta (perceptrone)	$\Delta w_j = \eta \cdot (d - o) \cdot x_j$
Discesa del gradiente	$w \leftarrow w - \eta \frac{\partial E}{\partial w}$
Entropia	$H(t) = - \sum_i p(i t) \log_2 p(i t)$
Equazione di Bellman	$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R + \gamma V^*(s')]$
Q-Learning update	$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

13.2 Tabella di confronto degli algoritmi di ricerca

Algoritmo	Informato	Completo	Ottimale	Tempo	Spazio
BFS	No	Sì	Sì*	$O(b^d)$	$O(b^d)$
DFS	No	No	No	$O(b^m)$	$O(bd)$
IDDFS	No	Sì	Sì*	$O(b^d)$	$O(bd)$
UCS	No	Sì	Sì	$O(b^{1+C^*/\epsilon})$	uguale
Greedy	Sì	No	No	$O(b^m)$	$O(b^m)$
A*	Sì	Sì	Sì	$O(b^d)$	$O(b^d)$
IDA*	Sì	Sì	Sì	$O(b^d)$	$O(bd)$
RBFS	Sì	Sì	Sì	varia	$O(bd)$

*Con costi uniformi.

13.3 Timeline storica dell'IA

- **1943** – McCulloch & Pitts: primo modello formale di neurone
- **1950** – Turing: Test di Turing; “Computing Machinery and Intelligence”

- **1956** – Dartmouth Conference: nasce l’IA come disciplina (McCarthy et al.)
- **1957** – Rosenblatt: perceptrone
- **1969** – Minsky & Papert: limiti del perceptrone (XOR) – primo AI winter
- **1986** – Rumelhart, Hinton, Williams: backpropagation per MLP
- **1992** – TD-Gammon: RL per backgammon a livello expert (Tesauro)
- **1997** – DeepBlue: scacchi contro Kasparov
- **2012** – AlexNet: deep learning per ImageNet (Hinton et al.)
- **2016** – AlphaGo: Go con reti neurali + MCTS (DeepMind)
- **2017** – Transformer: “Attention is All You Need” (Vaswani et al.)